

客观10-12

题量: 61 满分: 100

作答时间: 05-07 13:14 至 05-20 13:14

智能分析

86.9分

一. 单选题 (共38题, 60.9分)

1. (单选题)struct A {
A(int n):x(n) {}
int x = 1;
};
int main() {
A a1, a2(2);
cout<<a1()<<endl;
cout<<a2(3)<<endl;
}
若希望程序输出5和9, 那么下面定义哪个是正确的?

A. struct A {
A(int n = 1):x(n) {}
A(const A& rhs) { x=rhs.x+7; }
int operator() () const { return x+4;}
int x = 1;
};

B. struct A {
A(int n = 1):x(n) {}
int operator() () const { return x+4;}
int x = 1;
};

C. struct A {
A(int n = 1):x(n) {}
int operator() (int n = 0) const { return x+n+4;}
int x = 1;
};

D. struct A {
A(int n = 1):x(n) {}
int operator(int n= 0) const { return x+n+4;}
int x = 1;
};

我的答案: C 正确答案: C

答案解析:

2. (单选题)

class A {
public:
A(double aX,double aY):x(aX),y(aY) {}
private:
double x ;
double y;
};
若A类表示二维空间向量。现希望重载类A的点乘（内积）操作符(*)。
那么最合理的重载函数定义及实现是:

A.
class A {
public:

一. 单选题 (60.9分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38

https://mooc1.chaoxing.com/mooc-ans/mooc2/work/view?courseId=201642417&classId=93222363&cpi=352876303&workId=34815914&answe...

1/22

作业详情

```
};
inline double operator*(const A& b) {
    return x*b.x + y*b.y;
}

B.
class A {
public:
    friend double operator*(const A& a,const A& b);
    /*其它略*/
};
inline double operator*(const A& a,const A& b) {
    return a.x*b.x+a.y*b.y;
}

C.
class A {
public:
    double operator*(const A& b) const {
        return x *a.x + y*b.y;
    }
    /*其它略*/
};

D.
class A {
public:
    double operator*(const A& a,const A& b) const {
        return a.x*b.x + a.y*b.y;
    }
    /*其它略*/
};
```

一. 单选题 (60.9分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38

我的答案: D 正确答案: B  0 分

答案解析:

3. (单选题)若需要重载类A的后置--运算符，那么最合理的定义是:

```
A.
class A {
public:
    A& operator--( int );
};

B.
class A {
public:
    A& operator--( int ) const;
};

C.
class A {
public:
    A operator--( int );
};

D.
class A {
public:
    A operator--( int ) const;
};
```

一. 单选题 (60.9分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38

答案解析:

4. (单选题)若重载类A的前置++和后置++, 则正确的定义和实现是:

- A.
- ```
class A {
 A(int n):x(n) {}
 int & operator++() { return ++x;}
 int operator++(int) { int temp = x++; return temp; }
private:
 int x = 1;
};
```
- B.
- ```
class A {
    A(int n):x(n) {}
    A& operator++() { ++x; return *this;}
    A operator++(int) { ++x; return *this;; }
private:
    int x = 1;
};
```
- C.
- ```
class A {
 A(int n):x(n) {}
 A& operator++() { ++x; return *this;}
 A operator++(int) { ++x; return A(*this); }
private:
 int x = 1;
};
```
- D.
- ```
class A {
    A(int n):x(n) {}
    A& operator++() { ++x; return *this;}
    A operator++(int) { int temp=x; ++x; return A(temp); }
private:
    int x = 1;
};
```

我的答案: D 正确答案: D  1.6 分

答案解析:

5. (单选题)关于重载运算符时的要求, 下列哪些是不合理的?

- A. 不能重载作用域解析符、小数点和三目运算符
- B. 重载%运算符时, 建议同时重载%=运算符
- C. 重载单目运算符时, 应尽可能使用成员函数形式, 但使用自由函数形式也可以。
- D. 一个单目运算符可以由多个字符组成

我的答案: C 正确答案: C  1.6 分

6. (单选题)现需要开发一个第三方类库, 其中定义了类A和类B, 如下:

```
class A {
public:
    A( int n) : mp(new int(n)) { }
    ~A() { delete mp; }
```

```
};
class B {
public:
    B( int * p) : mp(p) { }
    ~B() { }
private:
    int * mp;
};
```

那么下面正确的说法是：

- A. 类A必须自定义拷贝构造函数和赋值函数，防止使用中造成内存泄漏
- B. 类B必须自定义拷贝构造函数和赋值函数，防止使用中造成内存泄漏
- C. 类A的析构函数实现应为： ~A() { if(mp) delete mp; }
- D. 类A的析构函数实现应为： ~A() { delete mp; mp=nullptr; }

E. 类A的析构函数实现应为：

```
~A() {
    if(mp) {
        delete mp; mp=nullptr;
    }
}
```

一. 单选题 (60.9分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38

我的答案: A 正确答案: A

✓

1.6 分

7. (单选题)下面程序的编译并执行，那么说法正确的是：

```
#include <iostream>
using namespace std;
class A {
public:
    A(int n=1):mVal(n) { }
    static void f() {
        static A a;
        A ** ps = new A*[3] { &a };
        for(int i=0;i<3;++i) {
            if( ps[i] ) cout<<(*ps[i]).mVal;
        }
    }
private:
    int mVal;
};
int main( ){
    A::f();
}
```

- A. 程序存在语法错误，不能通过编译
- B. 第一个数字输出1，其余两个数字不定
- C. 只输出一个数字1
- D. 输出111

我的答案: C 正确答案: C

✓

1.6 分

8. (单选题)

```
class A {
public:
    A(int n=0):mVal(n) { }
private:
    int mVal;
};
```

作业详情

```
A * p1 = new A( );           //2
A * p2 = new A(4);           //3
A * p3 = new A[2];           //4
A ** p4 = new A*[2] { };     //5
A ** p5 = new A*[2] { nullptr,new A(3) }; //6
/*略*/
}
```

那么没有在堆区中创建A类对象的语句是：

- A. //2和//4
- B. //2和//5
- C. //4和//5
- D. //5

我的答案: D

正确答案: D

✓

1.6 分

答案解析:

```
9. (单选题)struct A {
    int n = 1;
};
int main( )
{
    A * p1 = new A;    //1
    A * p2 = ::new A;  //2
    ::delete p2;       //3
    delete p1;         //4
}
```

那么下列说法错误的是：

- A. new A和::new A的语义是不一样的，只是本例中执行效果相同
- B. 建议程序员互换//3和//4的书写顺序
- C. 若struct A更复杂一些，执行上述main函数也可能有内存泄漏
- D. 将struct A改成class A，上述程序也能编译并执行

我的答案: B

正确答案: B

✓

1.6 分

```
10. (单选题)#include <iostream>
class A {
};
int main( ){
    A * p = new A;
    delete p;
    if(p==nullptr)
        std::cout<<1;
    else
        std::cout<<2;
}
```

程序的编译和执行情况是：

- A. 编译报错
- B. 编译通过，执行输出1
- C. 编译通过，执行输出2
- D. 编译通过，执行结果不确定，可能为1，也可能为2

一. 单选题 (60.9分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38

一. 单选题 (60.9分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38

```
11. (单选题)#include <iostream>
using namespace std;
class A {
public:
    A()          { cout<<1; }
    A(int n)     { cout<<2; }
    A(const A &) { cout<<3; }
};
int main() {
    A * p1 = new A;
    A * p2 = new A( );
    A * p3 = new A(5);
    A ** p4 = new A*(p3);
    A * p5 = new A(**p4);

    delete p5;
    delete p4;
    delete p3;
    delete p2;
    delete p1;
}
```

则程序的输出是:

- A. 1123
- B. 1223
- C. 11233
- D. 12233

我的答案: A 正确答案: A



1.6 分

答案解析:

12. (单选题)已知

```
struct A {
    int x;
};
struct B {
    int x;
    int y;
};
```

现共有3个A类和B类对象，但各类具体对象数量不确定。若想存储这3个对象，下面最合理的方法是:

- A. 定义数组， B objects[3]={ A(), A(), B() };
- B. 定义数组， B * objects[3]={ new A, new A, new B };
- C. 定义数组， void objects[3]={ A(), A(), B() };
- D. 定义数组， void * objects[3]={ new A, new A, new B };

我的答案: D 正确答案: D



1.6 分

13. (单选题)class Student

```
{
public:
    Student(const string& name) :mName(name) {}
private:
```

作业详情

现定义Team类，一个Team对象包含多个学生，学生人数由在构造Team对象时指定。那么下面定义最合理的是：

```
A. class Team {
public:
    Team(int n) ;
private:
    const int  mN = 10;
    Student  ms[mN];
};

B.
class Team {
public:
    Team(int n);
private:
    const int  mN = 10;
    Student  *ms[ ];
};

C. class Team {
public:
    Team(int n);
private:
    const int  mN = 10;
    Student  ** ms;
};

D. class Team {
public:
    Team(int n);
private:
    const int  mN = 10;
    Student  ** ms[ ];
};
```

一. 单选题 (60.9分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38

我的答案: C 正确答案: C

✓

1.6 分

```
14. (单选题)#include<iostream>
using namespace std;
struct A {
    int x = 1;
};
class B {
public:
    void f( A*& p) {           //1
        cout<<p->x;
        /* 其它略 */         //2
    }
};
int main( ) {
    B  pB;
    A * pA = new A;
    if( pA ) {                 //3
        pB.f(pA);
    }
    if( pA ) {                 //4
        delete pA;
    }
}
```

关于上边程序的说法，正确的是：

- A. //1中的参数类型，A*& 没有什么实际意义，就应该写成 A *
- B. //2中不能修改参数p的值
- C. 执行到//3时，pA一定非0，因此//3中if语句是多余的

作业详情

我的答案: D

正确答案: C

✖

0 分

答案解析:

15. (单选题)关于重载类A的operator new/operator new[], 错误的说法是:
- A. 重载的operator new/operator new[]和operator delete/operator delete[] 函数, 总是类方法
- B. 若重载了operator new, 应同时重载operator delete以及operator new[]和operator delete[]
- C. 类A中可以有5个以上的重载的operator new函数
- D. 在实现重载的operator new函数时, 不能再使用new运算符

我的答案: D

正确答案: D

✔

1.6 分

16. (单选题)shared_ptr是系统memory头文件中的一个类模板, 使用shared_ptr, 可以简化用户使用堆区对象, 进而减少程序中指针使用的错误。下面给出了一个简化版的shared_ptr类, 用以说明shared_ptr的实现及作用机制。

```
#include <iostream>
using namespace std;
class A {
public:
    A(int n = 1): mx(n) { cout << "A::A()" << endl; }
    void show() const { cout << "mx=" << mx << endl; }
    ~A() { cout << "A::~~A()" << endl; }
private:
    int mx;
};

class SharedPtr {
public:
    SharedPtr(A* p): mpA(p), mpused(new int(1)) {}
    SharedPtr(const SharedPtr& ptr): mpA(ptr.mpA), mpused(ptr.mpused)
    { ++*mpused; }
    SharedPtr& operator=(const SharedPtr& rhs) {
        if(&rhs != this) {
            release();
            mpA = rhs.mpA;
            mpused = rhs.mpused;
            ++*mpused;
        }
        return *this;
    }
    ~SharedPtr() { release(); }
    A& operator*() const { return *mpA; }
    A* operator->() const { return mpA; }
private:
    void release() {
        if(--(*mpused) == 0) {
            delete mpA;
            delete mpused;
            mpused = nullptr;
            mpA = nullptr;
        }
    }
private:
    A* mpA;
    int* mpused;
};

int main() {
    SharedPtr sptr1(new A(2));
```

一. 单选题 (60.9分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38

作业详情

```
sptr1->show();
sptr2->show();
sptr3->show();
}
```

从上边的示例代码，可以得出各种结论，其中说法错误的是？

- A. shared_ptr在实现时使用了引用计数技术
- B. shared_ptr只能共享指向堆区的指针
- C. 使用shared_ptr可以在形式上让程序员做到了：只管new而不用管delete
- D. 使用shared_ptr时，只能以对象形式使用，不能在堆区中创建shared_ptr类型对象

我的答案: D

正确答案: D

✓

1.6 分

17. (单选题)如下代码:

```
#include <iostream>
#include <memory>
using namespace std;
class Music {
public:
    Music(const string& str):name(str) {}
    string name;
};
class Person {
public:
    void listen(Music & music = Music("我的祖国")) {
        play(music);
    }
    void listen(const Music & music = string("我的祖国")) {
        play(music);
    }
    void listen(Music * music = nullptr) {
        if(music == nullptr)
            music = new Music("我的祖国");
        play(*music);
    }
    void listen(const Music * music = new Music("我的祖国")) {
        play(*music);
    }
    void listen(shared_ptr<Music> music = shared_ptr<Music>(new Music("我的祖国"))){
        play(*music);
    }
private:
    void play(const Music & m) { /* 略 */ }
};
int main() { /* 略 */ }
```

类Person中定义了5个重载的listen函数，问其中既合法又合理的有几个？

- A. 1
- B. 2
- C. 3
- D. 4

我的答案: C

正确答案: B

✗

0 分

答案解析:

18. (单选题)在f函数中创建局部数组a,并初始化各项为0,

```
int main() {
```

一. 单选题 (60.9分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38

作业详情

一. 单选题 (60.9分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38

对于N的定义，给出几种形式：

- 1. #define N 10
- 2. const int N =10;
- 3. enum { N =10};
- 4. namespace { int N =10; }

其中有几种是正确的？

- A. 1
- B. 2
- C. 3
- D. 4

我的答案: D 正确答案: D  1.6 分

```
19. (单选题)class B {
private:
    A *   pA;
    A **  ppA;
};
```

关于数据成员pA和ppA表示的含义，下列说法中正确的有共几个？

- 1.pA可以表示一个指针变量，该变量存放一个A类对象的地址.
- 2.pA可以表示一个指针变量，该变量存放一个数组的首地址，数组中各元素均为A类对象.
- 3.pA可以表示一个指针变量，该变量存放一个数组的首地址，数组中各元素存放A类对象的地址.
- 4.ppA可以表示一个指针变量，该指针指向另一个内存地址，该地址存放一个A类对象的地址.
- 5.ppA可以表示一个指针变量，该指针指向另一个内存地址，该地址存放一个数组的的首地址，数组中各元素均为A类对象.
- 6.ppA可以表示一个指针变量，该指针指向另一个内存地址，该地址存放一个数组的的首地址，数组中各元素均为A类对象的地址.

- A. 2
- B. 3
- C. 4
- D. 5

我的答案: C 正确答案: C  1.6 分

答案解析:

20. (单选题)在类定义时，可如下定义：

```
class B {
private:
    A   a[10];      //1
    A * pa[10];     //2
};
```

那么，关于//1和//2的说法，错误的是哪个？

- A. //1的形式，导致创建B类对象时，会创建10个数据都一样的A类对象.
- B. //1的形式，要求A类有无参构造函数或者在类B中提供自定义的构造函数.
- C. //2的形式，需要考虑是否自定义类B的拷贝构造、赋值等函数，而//1的形式不需要.
- D. //2的形式，可以实际只存3个A类对象，而//1的形式一定存满10个A类对象.

我的答案: B 正确答案: B  1.6 分

```
21. (单选题)对于如下形式的类B，
class B {
```

作业详情

一. 单选题 (60.9分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38

};
可以考虑使用引用计数，定义和实现相关类。关于引用计数的原因、目的的说法，错误的是：

A. 若类B不自定义拷贝构造、赋值函数，会进行“浅”拷贝和“浅”赋值，可能会产生逻辑错误；

B. 特定情况下，自定义拷贝构造、赋值函数也无法实现物理上的“深”拷贝和“深”赋值，但通过引用计数可做到逻辑意义上的深拷贝和赋值；

C. 类B自定义的拷贝构造、赋值函数，虽然会“深”拷贝和“深”赋值，但可能会效率低下；

D. 引用计数本质上就是共享，即在所有B类对象之间共享同一个A类对象，同时记录共享次数；

我的答案: D

正确答案: D

✓

1.6 分

22. (单选题)class B {
private:
 A * pa;
};
对于形如上边的类定义，可以考虑使用引用计数。
在具体实现时，既可以在A类记录引用次数，也可以在B类中记录引用次数。
但实际应用中，更多地采用在类B中记录，关于这么做的理由，错误的说法是：

- A. 类A可能已有其它的使用者，那么在类A记录引用次数，就需要修改类A代码，会影响类A的其它使用者.
- B. 这里类A的复用方式属于黑盒复用，意味着可能无法得到类A的源码，自然也无法修改类A的代码.
- C. 这里是类B复用类A，复用时不建议修改既有代码，那么修改类B就自然成为优先选择.
- D. 类B中有指针成员，通常总是要自定义类B的拷贝构造和赋值函数的，在类B这些函数的实现中记录引用次数更自然.

我的答案: D

正确答案: D

✓

1.6 分

23. (单选题)假设存在下面示意代码，则下面语句中属于强制(显式)类型转换的是:

```
class Foo{  
public:  
    Foo(int m=0) { }  
    /*略*/  
};  
Foo f;  
void func(Foo a) { /*略*/ }  
int n;
```

- A. f=10;
- B. f=Foo(10);
- C. func(10);
- D. n=10.0f;

我的答案: B

正确答案: B

✓

1.6 分

24. (单选题)假设存在下面示意代码，则下面语句中不正确的是：

```
class Mystr{  
public:  
    Mystr(const char* chrs) {}  
    // 略  
};  
class Foo{  
public:  
    Foo(const Mystr& str) : m_string(str) {}  
};
```

作业详情

```
Mystr m_string;
};
void func(Foo a){ a.output(); }

A. func("abc");
B. func(Mystr("abc"));
C. func(Foo("abc"));
D. Foo foo("abc");
```

我的答案: A 正确答案: A  1.6 分

答案解析:

25. (单选题)假设存在下面示意代码，则下面语句中正确的是：

```
class Mystr{
public:
    Mystr(const char* chrs) {}
    // 略
};
class Foo{
public:
    explicit Foo(const Mystr& str) : m_string(str) {}
    void output(void) { /*略*/ }
private:
    Mystr m_string;
};
void func(Foo a){ a.output(); }
```

A. func("abc");

B. func(Mystr("abc"));

C. func(Foo("abc"));

D. Foo foo = "abc";

我的答案: C 正确答案: C  1.6 分

答案解析:

26. (单选题)对于类A和类B，若A(B());能够编译通过，那么最可能是下面哪一种情况：

- A. A中有一个构造函数 A(const B*)
- B. A中有一个构造函数 A(const B&)
- C. B是A的友元类
- D. A中存在一个类型转换函数operator B()

我的答案: B 正确答案: B  1.6 分

27. (单选题)假设存在下面程序代码，则该程序的运行结果是：

```
class Foo {
public:
    explicit Foo() {}
    operator int() { return 10; }
    explicit operator double() { return 9.9; }
};
```

一. 单选题 (60.9分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38

作业详情

```
int i = f1;
double d1 = f1;
double d2 = (double)f1;
cout << i << " " << d1 << " " << d2 << endl;
return 0;
}
```

- A. 10 9.9 9.9
- B. 10 10 9.9
- C. 10 10 10
- D. 编译失败无法运行

我的答案: B

正确答案: B

✓

1.6 分

28. (单选题)下面哪个不应放在头文件中:

- A. namespace NS{ void f(int) {} };
- B. extern int f(int);
- C. class { protected: void f(int) {} };
- D. extern int count;

我的答案: A

正确答案: A

✓

1.6 分

29. (单选题)假设存在下面程序代码, 则该程序的运行结果是:

```
//main.cpp
namespace{ int num = 10; }
namespace Foo{ int num = 30; }
void func1(){ cout << num << endl; }
void func2();
int main() {
    func1();
    func2();
    cout<<num<<endl;
    using Foo::num;
    cout<<num<<endl;
}
//func.cpp
int num = 20;
void func2(){ cout << num << endl; }
```

- A. 10 10 10 30
- B. 10 20 20 30
- C. 10 20 10 30
- D. 编译失败无法运行

我的答案: C

正确答案: C

✓

1.6 分

30. (单选题)下面关于友元的说法, 不正确的是:

- A. 友元函数可直接访问类中私有成员
- B. 友元函数是成员函数, 它被声明在类体内
- C. 友元函数破坏封装性, 尽量少用

一. 单选题 (60.9分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38

我的答案: B 正确答案: B

✓

1.6 分

31. (单选题)下面关于友元的说法，正确的是：
- A. 一个类可以是另一个类的友元类，但两个类不能互为友元类
 - B. 一个类可以有多个友元函数，但不能同时有两个以上的友元类
 - C. 在类中声明友元函数时，指定其访问控制是不起作用的
 - D. 已知类A是类B的友元，类B是类C的友元，则类A一定是类C的友元

我的答案: C 正确答案: C

✓

1.6 分

32. (单选题)类Inner是类Outer的嵌套类，那么下列哪个不能成为类Outer的友元：
- A. 全局函数void f();
 - B. 全局整型变量 int var;
 - C. 类Inner;
 - D. 类Inner中的成员函数 void f();

我的答案: B 正确答案: B

✓

1.6 分

33. (单选题)class A {
public:
 static void f(A & a) {
 cout<<a.mx<<" "<<a.my<<" ";
 ++a.mx;
 ++a.my;
 }
 static void f(float val) {
 cout<<val;
 }
public:
 A(int x,int y=0):mx(x),my(y) {}
 explicit operator float() const { return sqrt(mx*mx+my*my); }
private:
 int mx;
 int my;
};
int main() {
 A a(2,3);
 A::f(a);
 A::f(float(a));
}
的输出结果是：

- A. 2 0 2
- B. 2 3 4
- C. 2 3 5
- D. 3 4 5

我的答案: C 正确答案: C

✓

1.6 分

一. 单选题 (60.9分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38

作业详情

```
namespace China {
    class A {
    public:
        void f() const { cout<<"1"; }
    };
    namespace University {
        class B {
        public:
            void g() const { cout<<"2"; }
        };
    }
}
int main() {
    using namespace China;
    B b;    //1
    b.g();
    return 0;
}
```

为了使程序正确执行，可通过修改//1所在行的代码来实现。下面哪种修改是错误的：

- A. using namespace China::University; B b;
- B. using namespace University; B b;
- C. ::University::B b;
- D. University::B b;

一. 单选题 (60.9分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38

我的答案: C

正确答案: C

✓

1.6 分

```
35. (单选题)class B;
class A {
public:
    A(int n):ma(n) {}
    void f(B &a);
private:
    int ma;
};
class B {
public:
    B(int n):mb(n) {}
    void g(A&b);
private:
    int mb;
};

void A::f(B&b) {
    cout<<b.mb<<endl;
}
void B::f(A&a) {
    cout<<a.ma<<endl;
}
```

为了使上述代码编译通过，可通过：

- A. 在A类中，添加 friend class B;
- B. 在B类中，添加 friend class A;
- C. 在A类中，添加 friend class B; 且在B类中，添加 friend class A;
- D. 在A类中，添加 friend B::mb; 且在B类中，添加 friend A::ma;

我的答案: A

正确答案: C

✗

0 分

作业详情

- A. 流是对数据移动方式的一种抽象
- B. 字节流、字符流、文件流、内存流等是针对不同数据特点对流的简单分类
- C. cout,cin是系统预定义的流函数
- D. 使用缓冲区可以有效提高流的处理效率

我的答案: C 正确答案: C  1.6 分

37. (单选题)关于嵌套类的说法，错误的是：

- A. 内层的嵌套类只能具有public访问控制
- B. 嵌套类名字的可见范围(scope)，类似嵌套名字空间中名字的可见范围
- C. 外层的嵌套类与内层的嵌套类之间，不是包含和被包含关系
- D. 嵌套类也可以看做实现信息隐蔽的一种特殊机制

我的答案: A 正确答案: A  1.6 分

```
38. (单选题)namespace My {
    class Computer {
        //略
    };
    namespace Book {
        class Novel {
            //略
        };
    }
}
int main() {
    //创建Novel对象
}
```

若需要在main函数中创建局部Novel对象，下面给出的多种语句块，有几个是合法的？

- 1. using namespace My; Book::Novel novel;
- 2. using namespace My::Book; Novel novel;
- 3. using namesapce My; ::Book::Novel novel;
- 4. ::My::Book::Novel novel;
- 5. My::Book::Novel novel;
- 6. using My::Book::Novel; Novel novel;
- 7. using namespace My; using Book::Novel; Novel novel;

- A. 4个
- B. 5个
- C. 6个
- D. 7个

我的答案: C 正确答案: C  1.7 分

答案解析:

一. 单选题 (60.9分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38

二. 多选题 (共7题， 11.9分)

39. (多选题)已知

```
class B {
public:
```


作业详情

```
void f3();
};
class A {
    A(B& b):mpB(&b) {}
    // 待添加
private:
    B * mpB;
};
int main() {
    B b;
    A a(b);
    //通过对象a, 访问其中的B类对象
    //...
}
```

若希望在main函数中，可以通过a对象访问其内部的B类对象，可以采用的方法是：

- A. 在A类中添加成员： B * A::getB() const { return mpB; }，在main中： a.getB()->f1(); a.getB()->f2();
- B. 在A类中添加成员： B * operator->() const { return mpB; }，在main中： a->f1(); a->f2();
- C. 在A类中添加成员： B * operator->() const { return mpB; }，在main中： a.f1(); a.f2();
- D. 在A类中添加成员： B & operator &() const { return mpB; }，在main中： a.f1(); a.f2();

一. 单选题 (60.9分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38

我的答案: ABCD

正确答案: AB

✖

0分

40. (多选题)

已知有如下的类A定义：

```
class A {
public:
    int & operator[ ](int index)      { return ms[index]; }
    int  operator[ ](int index) const { return ms[index]; }
private:
    int  ms[5] = { 1,2,3,4,5 };
};
```

若需要将类A中数据成员ms的类型改为类型B，即改为 B ms[5] = {1,2,3,4,5};
那么，说法正确的是：

- A. 应确保类B有无参构造函数
- B. 类B必然存在自定义的构造函数
- C. 重载的operator[]函数，应改为:
B & operator[](int index) { return ms[index]; }
const B operator[](int index) const { return ms[index]; }
- D. 重载的operator[]函数，应改为:
B & operator[](int index) { return ms[index]; }
B operator[](int index) const { return ms[index]; }
- E. 重载的operator[]函数，应改为:
B & operator[](int index) { return ms[index]; }
const B & operator[](int index) const { return ms[index]; }
- F. 重载的operator[]函数，应改为:
B operator[](int index) { return ms[index]; }
const B operator[](int index) const { return ms[index]; }

我的答案: BCDEF

正确答案: BE

✖

0分

```
class A {
public:
    B    bs[N];
};
```

那么下面的哪些说法是合理的:

- A.
a.h中添加#include "b.h",可以避免各文件在需要#include "a.h"时头文件的include顺序限制
- B.
类A中的N必须是编译期常量
- C. 创建A类对象时, 会创建N个B类对象
- D. 释放A类对象时, 会析构N个B类对象

我的答案: ABCD 正确答案: ABCD  1.7 分

答案解析:

一. 单选题 (60.9分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38

42. (多选题)

编译并执行下面的代码时,

```
#include <iostream>
using namespace std;
class A {
public:
    A (int n=1):mVal(n) {}
    int mVal;
};
int main() {
    A * ps[2] = { new A(2) };           //1
    cout<<ps[0]->mVal<<ps[1]->mVal;    //2
    delete ps[1];                       //3
    delete ps[0];                       //4
}
```

那么正确的是:

- A. 编译错误, 无法生成exe文件
- B. 编译正确, 可以执行, 输出 21
- C. 编译正确, 可以执行, 输出 23
- D. 编译正确, 可以执行, 但//1存在运行时错误
- E. 编译正确, 可以执行, 但//2存在运行时错误
- F. 编译正确, 可以执行, 但//3存在运行时错误
- G. 编译正确, 可以执行, 但//4存在运行时错误

我的答案: E 正确答案: E  1.7 分

43. (多选题)

```
class A {
public:
    static void f() {
        A * ps[2] = new A[2]{}; //1
        A ** pps = new A*[2]{}; //2
        delete[2] ps;           //3
        delete ps[0]; delete ps[1]; //4
        delete[ ] ps;           //5
        delete[ ] pps;          //6
        delete pps[ ];          //7
    }
};
```

作业详情

哪些行是错误的：

- A. //1所在行
- B. //2所在行
- C. //3所在行
- D. //4所在行
- E. //5所在行
- F. //6所在行
- G. //7所在行

我的答案: ACDEG

正确答案: ACDEG

✓

1.7 分

答案解析:

一. 单选题 (60.9分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38

44. (多选题)

```
struct A {
    A() { cout<<1;}
    void * operator new(size_t s)
    {
        cout<<s<<endl; //1
        x = 8; //2
        return ::new A( ); //3
        return new A( ); //4
        return new char[s]; //5
        return ::new char[s]; //6
        return ::operator new(s); //7
    }
    int x;
};
```

3,4,5,6,7行只可以保留一行，那么1-7中不合理的是：

- A. //1
- B. //2
- C. //3
- D. //4
- E. //5
- F. //6
- G. //7

我的答案: BDF

正确答案: BD

✗

0 分

答案解析:

45. (多选题)在讨论共享指针时，其实针对的是B类中有A类型的指针数据成员的形式，即如下形式：

```
class B {
    A * mpA;
};
```

那么说法正确的是：

- A. 若B类中只有A类型的对象成员，没有必要讨论共享的问题
- B. 实现共享，可以在A类中定义计数变量，也可以在B类中定义计数变量
- C. 实现共享时，若在B类中定义计数变量，那么该变量需要定义成static整型，或指向整型的指针

我的答案: AB

正确答案: AB

✓

1.7 分

三. 判断题 (共16题, 27.2分)

46. (判断题) 在实现operator delete重载函数时, 应保证函数执行过程中不会出现运行时错误, 以免造成内存泄漏

- A. 对
- B. 错

我的答案: 对

正确答案: 对

✓

1.7 分

答案解析:

47. (判断题) 无论是在栈区还是在堆区创建的数组, 其内存空间一定是连续的, 每个元素所占的存储空间大小一定是一样的。

- A. 对
- B. 错

我的答案: 对

正确答案: 对

✓

1.7 分

答案解析:

48. (判断题) 在堆区中, 可以动态创建一个对象, 但不能动态创建一个整数。

- A. 对
- B. 错

我的答案: 错

正确答案: 错

✓

1.7 分

49. (判断题) 使用new操作符创建A类对象, 一定会调用A类的无参构造函数。

- A. 对
- B. 错

我的答案: 错

正确答案: 错

✓

1.7 分

50. (判断题) 相对于new操作符, C语言的malloc函数在语义上更接近operator new 操作符

- A. 对
- B. 错

我的答案: 对

正确答案: 对

✓

1.7 分

51. (判断题) 对于语句A* pA = new A;, 在运行时可能会产生错误, 但delete pA;一般不会产生运行错误, 也不应该产生。

- A. 对

一. 单选题 (60.9分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38

作业详情

我的答案: 对

正确答案: 对

✓

1.7 分

52. (判断题)类A中可以自定义重载的operator new运算符，但这些重载函数一定都是类方法

- A. 对
- B. 错

我的答案: 对

正确答案: 对

✓

1.7 分

53. (判断题)使用delete释放对象时，应先通过if语句判定指针是否为空。

- A. 对
- B. 错

我的答案: 错

正确答案: 错

✓

1.7 分

54. (判断题)C++中只能在堆区内存上使用定位分配。

- A. 对
- B. 错

我的答案: 错

正确答案: 错

✓

1.7 分

55. (判断题)使用delete[]释放数组时，需要释放的总字节数在编译时即可确定，因此使用语句delete[] p;，而不是delete[5] p;形式。

- A. 对
- B. 错

我的答案: 错

正确答案: 错

✓

1.7 分

答案解析:

56. (判断题)代码double dval; float *pi=&dval;中会发生自动(隐式)类型转换，因此不存在语法错误。

- A. 对
- B. 错

我的答案: 错

正确答案: 错

✓

1.7 分

57. (判断题)单参构造函数一定可以被用来执行自动(隐式)类型转换。

- A. 对
- B. 错

我的答案: 错

正确答案: 错

✓

1.7 分

一. 单选题 (60.9分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38

作业详情

- A. 对
- B. 错

我的答案: 对

正确答案: 对

✓

1.7 分

59. (判断题)名字空间是C++中一种解决命名冲突的机制。

- A. 对
- B. 错

我的答案: 对

正确答案: 对

✓

1.7 分

60. (判断题)一个类只能访问其所在名字空间中的内容，不能访问其它名字空间中的内容。

- A. 对
- B. 错

我的答案: 错

正确答案: 错

✓

1.7 分

61. (判断题)类Outer中有嵌套类Inner，那么sizeof(Outer)的值大于等于sizeof(Inner)的值。

- A. 对
- B. 错

我的答案: 错

正确答案: 错

✓

1.7 分

一. 单选题 (60.9分)

1	2	3
6	7	8
11	12	13
16	17	18
21	22	23
26	27	28
31	32	33
36	37	38